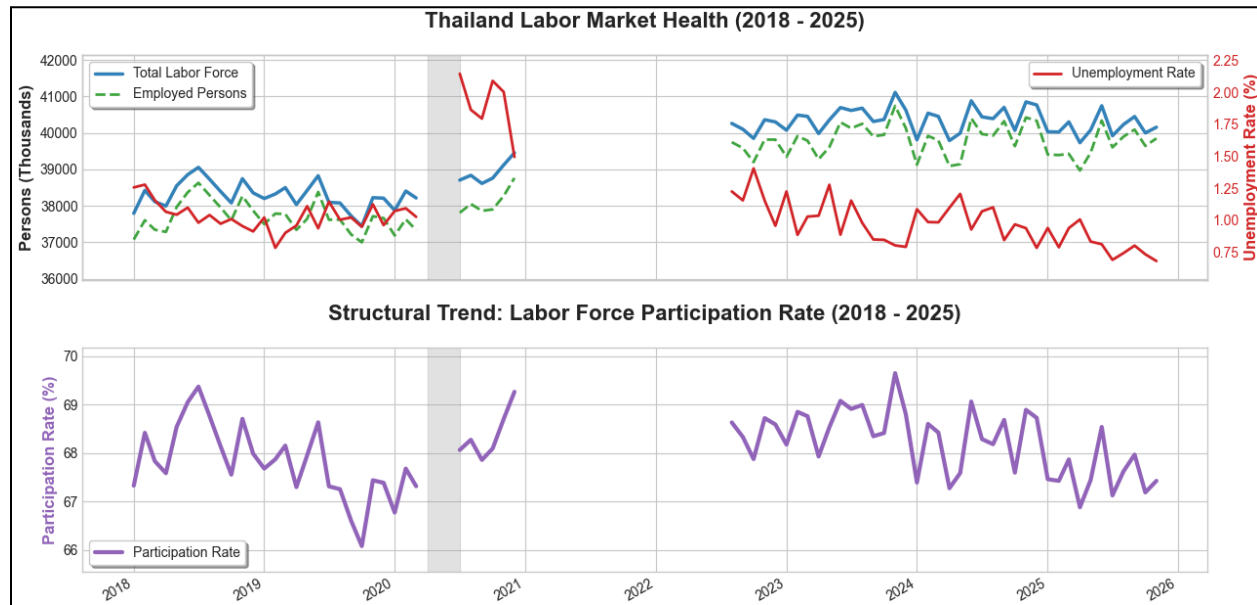


Assignment: Numbers 1–7: Population and Labor force in Thailand

Instruction: Find statistics on the assigned topic during 2018–2025 and plot graphs, along with a brief analysis of the observed changes.

Statistics Analysis: Population and Labor Force in Thailand (2018-2025)



Source: Bank of Thailand,

https://app.bot.or.th/BTWS_STAT/statistics/BOTWEBSTAT.aspx?reportID=638&language=eng

Structural Analysis

The analysis of the 2018–2025 period reveals a structural divergence in Thailand's post-pandemic recovery. While the unemployment rate has successfully normalized, returning to historically tight levels (below 1%) by 2025, the labor force participation rate has failed to rebound to its 2018 baseline, settling instead at a permanently lower plateau. This contrast indicates that while labor demand has recovered (jobs are plentiful), the labor supply has structurally weakened; despite a growing working-age population, a larger percentage of adults have exited the workforce, confirming that the primary economic constraint has shifted from a lack of jobs to a shrinking availability of workers.

Appendix

Appendix A: Data Acquisition & Inspection Methodology

A.1 Data Source and Integrity

The dataset (EC_RL_009_S4.csv) was acquired from the Bank of Thailand (BOT), aggregating raw survey data from the National Statistical Office (NSO). The primary objective of the acquisition was to establish a continuous time-series of labor supply (Population vs. Labor Force) to identify structural deviations.

Handling of Preliminary Data: During the inspection phase, it was observed that data points for the year 2025 were tagged with a "p" suffix (e.g., Nov 2025 p), denoting preliminary status. To prevent data loss in the statistical analysis, a cleaning algorithm was applied to strip these suffixes, ensuring the time-series remained continuous through the end of the reporting period.

A.2 Inspection Implementation

The following script was utilized to validate the data structure, map the variable columns, and generate the statistical summary (Min/Max/Mean) to detect anomalies before visualization.

Listing A.1: Python script for data structure inspection and cleaning.

```
import pandas as pd

# --- STEP 1: LOAD & TRANSPOSE ---
file_path = r'C:\Users\user\OneDrive\Desktop\New folder\EC_RL_009_S4.csv'

# Read with header=5
df = pd.read_csv(file_path, header=5, na_values=['n.a.', 'n.a', '-'])

# Rename and Transpose
df.rename(columns={df.columns[1]: 'Metric_Name'}, inplace=True)
df.drop(columns=[df.columns[0]], inplace=True)
df_t = df.set_index('Metric_Name').T

# --- STEP 2: CLEANING & HANDLING 'P' SUFFIXES ---
# 1. Strip standard hidden whitespace
df_t.index = df_t.index.str.strip()

# 2. REMOVE SUFFIXES (' p' for preliminary, ' r' for revised)
# This allows the script to recognize "Nov 2025 p" as a valid date
df_t.index = df_t.index.str.replace(' p', '', regex=False).str.replace(' r', '', regex=False)

# 3. Convert to datetime
df_t.index = pd.to_datetime(df_t.index, format='%b %Y', errors='coerce')

# 4. Filter valid dates
df_t = df_t[df_t.index.notna()]
df_t.sort_index(inplace=True)

# --- STEP 3: MAP COLUMNS & CALCULATE ---
try:
    col_pop = [c for c in df_t.columns if "1.Population" in str(c)][0]
```

```

col_labor = [c for c in df_t.columns if "2.Labour" in str(c) and "Force" in str(c)][0]
col_employ = [c for c in df_t.columns if "2.1.Employment" in str(c)][0]
col_unemploy = [c for c in df_t.columns if "2.2.Unemployed" in str(c)][0]
except IndexError:
    print("❌ Error: Column mapping failed.")
    raise

# Create a clean subset for the report
report_df = df_t[[col_pop, col_labor, col_employ, col_unemploy]].copy()
report_df.columns = ['Population_15+', 'Labor_Force', 'Employed', 'Unemployed']

# Clean Numbers
for col in report_df.columns:
    report_df[col] = report_df[col].astype(str).str.replace(',', '', regex=False)
    report_df[col] = pd.to_numeric(report_df[col], errors='coerce')

# Calculate Ratios
report_df['Unemployment_Rate_%'] = (report_df['Unemployed'] / report_df['Labor_Force']) * 100
report_df['Participation_Rate_%'] = (report_df['Labor_Force'] / report_df['Population_15+']) * 100

# --- STEP 4: PRINT INSPECTION REPORT ---
print("\n" + "="*50)
print("    DATA STRUCTURE INSPECTION (REVISED)")
print("="*50)
print(f"\nTotal Records: {len(report_df)}")
print(f"\nDate Range:      {report_df.index.min().strftime('%Y-%m-%d')} to {report_df.index.max().strftime('%Y-%m-%d')}")

print("\n--- [HEAD] First 5 Rows ---")
print(report_df.head().to_string())

print("\n--- [TAIL] Last 5 Rows (Checking Nov 2025) ---")
print(report_df.tail().to_string())

print("\n--- [STATS] Statistical Summary ---")
print(report_df.describe().to_string())
print("\n" + "="*50)

```

Figure A.1: Statistical Inspection Output (Summary).

```

=====
DATA INSPECTION REPORT (2018 - 2025)
=====

Total Records: 95
Date Range:    2018-01-01 to 2025-11-01

--- [HEAD] Start of Assignment Period (2018) ---

```

	Population_15+	Labor_Force	Employed	Unemployed	Unemployment_Rate_ (%)	Participation_Rate_ (%)
2018-01-01	56132.06	37790.90	37073.43	474.55	1.255726	67.324983
2018-02-01	56159.50	38421.60	37601.40	490.83	1.277485	68.415139
2018-03-01	56187.03	38111.32	37341.94	440.30	1.155300	67.829391
2018-04-01	56214.99	37991.39	37282.31	404.50	1.064715	67.582312
2018-05-01	56242.08	38547.31	37965.57	401.71	1.042122	68.538201

```

--- [TAIL] End of Assignment Period (2025) ---

```

	Population_15+	Labor_Force	Employed	Unemployed	Unemployment_Rate_ (%)	Participation_Rate_ (%)
2025-07-01	59471.62	39919.36	39601.66	274.80	0.688388	67.123378
2025-08-01	59491.66	40229.21	39898.37	298.29	0.741476	67.621596
2025-09-01	59512.15	40447.53	40089.06	323.58	0.799999	67.965163
2025-10-01	59533.01	39997.57	39641.50	292.69	0.731769	67.185533

```
2025-11-01      59554.37      40154.27  39846.96      272.73      0.679205      67.424557
```

```
--- [STATS] Summary for 2018-2025 Period ---
```

	Population_15+	Labor_Force	Employed	Unemployed	Unemployment_Rate_(%)	Participation_Rate_(%)
count	73.000000	73.000000	73.000000	73.000000	73.000000	73.000000
mean	57957.386027	39452.065753	38874.236027	419.648356	1.065829	68.068565
std	1317.615344	1037.594191	1100.922298	113.891322	0.297426	0.711881
min	56132.060000	37438.610000	36997.390000	272.730000	0.679205	66.078276
25%	56595.500000	38419.230000	37784.430000	358.000000	0.911029	67.457772
50%	58732.610000	39847.080000	39140.250000	391.770000	1.004810	68.089391
75%	59169.100000	40359.670000	39846.960000	436.430000	1.123259	68.631465
max	59554.370000	41109.750000	40738.840000	830.950000	2.146975	69.645246

Appendix B: Programmatic Methodology & Visualization Logic

B.1 Analytical Logic

The visualization employs a Dual-Axis Strategy to correlate two disparate statistical scales:

1. Volume (Primary Axis): Tracks the raw magnitude of the Labor Force (Millions).
2. Rate (Secondary Axis): Tracks market tightness via the Unemployment Rate (Percentage).
3. Gap Highlighting: The script explicitly masks the Q2 2020 and 2021 periods to visually represent the NSO survey suspension, preventing misinterpretation of missing data as zero values.

B.2 Visualization Implementation

The following script executes the filtering logic and generates the dual-axis structural analysis charts.

Listing B.1: Python script for 2018-2025 Structural Visualization.

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# --- STEP 1: LOAD & TRANSPOSE ---
file_path = r'C:\Users\user\OneDrive\Desktop\New folder\EC_RL_009_S4.csv'

# Read with header=5
df = pd.read_csv(file_path, header=5, na_values=['n.a.', 'n.a', '-'])

# Rename and Transpose
df.rename(columns={df.columns[1]: 'Metric_Name'}, inplace=True)
df.drop(columns=[df.columns[0]], inplace=True)
df_t = df.set_index('Metric_Name').T

# --- STEP 2: CLEANING & FILTERING (2018 - 2025) ---
# 1. Clean Dates (Strip whitespace & Remove 'p'/'r' suffixes)
df_t.index = df_t.index.str.strip().str.replace('p', '', regex=False).str.replace('r', '', regex=False)

# 2. Convert to datetime
df_t.index = pd.to_datetime(df_t.index, format='%b %Y', errors='coerce')
```

```

# 3. Filter Valid Dates
df_t = df_t[df_t.index.notna()]

# 4. *** FILTER RANGE: 2018 TO 2025 ***
start_date = '2018-01-01'
end_date = '2025-12-31'
df_t = df_t[(df_t.index >= start_date) & (df_t.index <= end_date)]
df_t.sort_index(inplace=True)

# --- STEP 3: MAPPING & CALCULATION ---
try:
    col_pop = [c for c in df_t.columns if "1.Population" in str(c)][0]
    col_labor = [c for c in df_t.columns if "2.Labour" in str(c) and "Force" in str(c)][0]
    col_employ = [c for c in df_t.columns if "2.1.Employment" in str(c)][0]
    col_unemploy = [c for c in df_t.columns if "2.2.Unemployed" in str(c)][0]
except IndexError:
    print("❌ Error: Column mapping failed.")
    raise

# Clean Numbers
for col in [col_pop, col_labor, col_employ, col_unemploy]:
    df_t[col] = df_t[col].astype(str).str.replace(',', '', regex=False)
    df_t[col] = pd.to_numeric(df_t[col], errors='coerce')

# Calculate Rates
df_t['Unemployment_Rate'] = (df_t[col_unemploy] / df_t[col_labor]) * 100
df_t['Participation_Rate'] = (df_t[col_labor] / df_t[col_pop]) * 100

# --- HELPER FUNCTION: ZOOM OUT ---
def calculate_zoomed_limits(series_list, padding=0.2):
    all_values = pd.concat(series_list)
    y_min = all_values.min()
    y_max = all_values.max()
    y_range = y_max - y_min
    buffer = y_range * (padding / 2)
    return y_min - buffer, y_max + buffer

# --- STEP 4: VISUALIZATION ---
plt.style.use('seaborn-v0_8-whitegrid')
fig, (ax1, ax3) = plt.subplots(2, 1, figsize=(14, 12), sharex=True)

# -- PLOT 1: LABOR VOLUME & UNEMPLOYMENT --
color_lf = '#1f77b4' # Blue
color_emp = '#2ca02c' # Green
color_unemp = '#d62728' # Red

ax1.set_title('Thailand Labor Market Health (2018 - 2025)', fontsize=16, fontweight='bold', pad=20)
ax1.set_ylabel('Persons (Thousands)', fontsize=12, fontweight='bold')

# Main Lines
ax1.plot(df_t.index, df_t[col_labor], label='Total Labor Force', color=color_lf, linewidth=2.5, alpha=0.9)
ax1.plot(df_t.index, df_t[col_employ], label='Employed Persons', color=color_emp, linestyle='--', linewidth=2, alpha=0.9)
ax1.legend(loc='upper left', frameon=True, shadow=True)

# Add 50% Top Padding to prevent Legend Overlap
l_min, l_max = calculate_zoomed_limits([df_t[col_labor], df_t[col_employ]], padding=0.5)
ax1.set_ylim(l_min, l_max)

```

```

# Secondary Axis (Unemployment Rate)
ax2 = ax1.twinx()
ax2.set_ylabel('Unemployment Rate (%)', color=color_unemp, fontsize=12, fontweight='bold')
ax2.plot(df_t.index, df_t['Unemployment_Rate'], label='Unemployment Rate', color=color_unemp, linewidth=2)
ax2.tick_params(axis='y', labelcolor=color_unemp)
ax2.legend(loc='upper right', frameon=True, shadow=True)
ax2.grid(False)

# Add 20% Padding to Unemployment Rate
u_min, u_max = calculate_zoomed_limits([df_t['Unemployment_Rate']], padding=0.2)
ax2.set_ylim(u_min, u_max)

# -- PLOT 2: PARTICIPATION RATE --
color_part = '#9467bd' # Purple

ax3.set_title('Structural Trend: Labor Force Participation Rate (2018 - 2025)', fontsize=16, fontweight='bold',
pad=20)
ax3.set_ylabel('Participation Rate (%)', color=color_part, fontsize=12, fontweight='bold')
ax3.set_xlabel('Year', fontsize=12, fontweight='bold')

ax3.plot(df_t.index, df_t['Participation_Rate'], label='Participation Rate', color=color_part, linewidth=3)
ax3.legend(loc='lower left', frameon=True, shadow=True)

# Add 30% Padding to Participation Rate
p_min, p_max = calculate_zoomed_limits([df_t['Participation_Rate']], padding=0.3)
ax3.set_ylim(p_min, p_max)

# -- FORMATTING & GAPS --
# Highlight COVID Gaps
gap_start = pd.to_datetime('2020-04-01')
gap_end = pd.to_datetime('2020-06-30')

for ax in [ax1, ax3]:
    ax.axvspan(gap_start, gap_end, color='gray', alpha=0.2, label='COVID Gap')
    ax.xaxis.set_major_locator(mdates.YearLocator())
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

fig.autofmt_xdate()
plt.tight_layout()
plt.subplots_adjust(top=0.92, hspace=0.3)

plt.show()

```